

Problem 1b: Understand BERT Inputs

Q) Look at the tokenized inputs from two code cells above. The inputs are represented as a dict with three keys: `input_ids`, `token_type_id`, and `attention_mask`. What do each of those three inputs represent?

- `input_ids`: These are numerical representations or indices of the tokens for the corresponding sequences. are the primary form for BERT to process the text.
- `token_type_ids`: These indicates to which sentence a token belongs to when multiple sentences exist in a sequence. They help BERT to distinguish different segments in tasks such as question answering, where a pair of sentences (e.g. <Question, Answer>) is used in a single sequence.
- `attention_mask`: This is a binary indicator that tells the model whether the token should be attended to and which are padding. When batching sequences, sequences with shorter length are padded, and this attention indicator ensures the model focuses on meaningful parts while ignoring the padded tokens.

Problem 1c: Understand BERT Hyperparameters

Q) Describe how hyperparameter tuning was performed for the GLUE benchmark consulting original BERT paper as well as official BERT Github repository under the heading "**New March 11th, 2020: Smaller BERT Models.**"

Hyperparameters tuning for the GLUE benchmark was done by finding the best performed "batch size" and "learning rate" on the development (validation) set from the below predefined sets with 4-epoch training.

- batch sizes: [8, 16, 32, 64, 128]
- learning rates: [3e-4, 1e-4, 5e-5, 3e-5]

Problem 3a: Train Models

Q) Report the highest validation accuracy attained in each of your two training runs, as well as the hyperparameters used in those trials.

	VALIDATION ACCURACY	LEARNING RATE	BATCH SIZE
Without BitFit	0.889	0.0003	32
With BitFit	0.628	0.0003	16

Problem 3b: Test Models and Report Results

Q) For each of your two training runs, please test the model that attained the best validation accuracy across all hyperparameter tuning trials. Once testing is complete, please report your results. Finally, please comment on your results. How do they compare to the results reported by Zaken et al. (2020)? What does this say about BitFit and its applicability to other pre-trained Transformers?

Results presented by Zaken et al. show that the performance of the model with and without BitFit on GLUE tasks except for the WNLI task is similar. However, in our experimentations on `imdb` binary sentiment classification task, the accuracy with BitFit is less than without BitFit by around 25%pt. I believe this is because the BERT_base model (110M) was used in Zaken et al.'s paper, whereas the BERT_tiny (4.4M) was used in our experiments. BERT_tiny has fewer parameters and more limited capacity compared to BERT_base, which could be restrictive in fine-tuning. Therefore, advantages of BitFit depend heavily on the size of the underlying pre-trained Transformer.

	# TRAINABLE PARAMETERS	TEST ACCURACY
Without BitFit	4386178	0.865
With BitFit	3074	0.624